MODULE 35 · FOUNDATIONS AND EXAM SKILLS - ALL DOMAINS

Reading Foundry Python code

Four questions for any snippet, the client shapes, the parameters that decide answers, JSON payloads, and the bugs the exam plants.

DOMAIN Exam skills WEIGHT - MODULE 35 of 39

admin@sthithapragna.com

Read four things: which client, which operation, which parameters, and which one is wrong.

What each question tells you

QUESTION	WHAT IT REVEALS
Which client?	The service, the endpoint, and the auth method
Which operation?	The feature being tested: responses, images, search, agents
Which parameters?	The setting the question turns on
Which is wrong?	The single planted fault, usually a name, a value, or an order

> ON THE EXAM
>
> Most snippets contain one deliberate mistake. Everything else is scenery.

The three openings you'll see

```python
# 1. Foundry project client, then its OpenAI client
project = AIProjectClient(endpoint=endpoint, credential=DefaultAzureCredential())
client = project.get_openai_client()

# 2. OpenAI SDK straight at the v1 endpoint, keyless
client = OpenAI(base_url=f"{endpoint}/openai/v1",
                api_key=get_bearer_token_provider(
                    DefaultAzureCredential(), "https://ai.azure.com/.default"))

# 3. Key-based client, which production guidance discourages
client = OpenAI(base_url=f"{endpoint}/openai/v1", api_key=os.environ["KEY"])
```

ON THE EXAM

DefaultAzureCredential means Entra ID. An api_key string means keys, which questions about security usually penalise.

What the first two lines tell you

- **AIProjectClient**: a Foundry project, so agents, connections and evaluations are in scope.
- **base_url ending /openai/v1**: the v1 API, so no dated api-version.
- **SearchClient** or **SearchIndexClient**: Azure AI Search, not a model call.
- **api-version in a URL**: a REST call, such as Language or Content Understanding.
- **DefaultAzureCredential**: Entra ID, which needs a role assignment somewhere.

> ON THE EXAM
>
> Naming the service in the first two lines removes most wrong options before you read the rest.

Agents, conversations and responses

```
agent = project.agents.create_version(
    agent_name="returns-assistant",
    definition=PromptAgentDefinition(model="chat-prod",
                                     instructions=text, tools=[order_tool]))

conversation = openai.conversations.create()

reply = openai.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": "returns-assistant",
                                    "type": "agent_reference"}},
    input="Where is order ORD-10442?")
```

ON THE EXAM

create_version defines an agent; responses.create runs one; conversation= carries the history.

The settings questions turn on

PARAMETER	WHAT IT DECIDES
`model`	The deployment name, not the model name
`store, previous_response_id, conversation`	Whether and how history is kept
`max_completion_tokens / max_output_tokens`	The cap reasoning models accept
`text.format with strict`	A guaranteed JSON shape
`query_type, k, semantic_configuration`	How a search query ranks
`context, inputs, outputs`	How often a skill runs, and where output lands

> ON THE EXAM
>
> If a question shows a parameter list, the answer is usually one line of it.

In Azure code, **model=** takes a **deployment name**. A model name that was never deployed is the most common planted bug.

Hybrid and semantic in the Search SDK

```
results = search.search(
    search_text=question,
    vector_queries=[VectorizableTextQuery(text=question,
        k_nearest_neighbors=50, fields="content_vector")],
    query_type="semantic",
    semantic_configuration_name="default",
    filter="region eq 'UK'",
    select=["title", "content", "url"],
    top=10)
```

ON THE EXAM

search_text plus vector_queries is hybrid. k of 50 feeds the semantic ranker; top of 10 is what you return.

Reading the JSON questions

- **@odata.type** names the skill in a skillset: OCR, split, entity recognition, custom Web API.
- **kind** names the task in a Language REST call: EntityRecognition, PiiEntityRecognition.
- **context** in a skill decides how often it runs; paths start at /document.
- **policyKind** in a PII call chooses the redaction policy.
- A custom skill's response must carry **recordId**, **errors** and **warnings**.

> ON THE EXAM
>
> JSON questions are the same four questions: which service, which operation, which parameters, which is wrong.

What usually goes wrong in exam code

SYMPTOM IN THE QUESTION	FAULT IN THE CODE
404 not found	model set to a model name, not a deployment
Unsupported parameter error	temperature or max_tokens on a reasoning model
Code expects a URL for an image	GPT-image models return base64 only
Nothing traced at all	Tracing variable set after instrument()
Custom skill output ignored	Response missing recordId, errors or warnings
Ranker seems to do nothing	k too small, or no semantic configuration

ON THE EXAM

Six faults cover most code questions. Each one has appeared in an earlier module of this course.

Reading a snippet under time pressure

- Read the **first two lines** and the **call being made**. Ignore the rest at first.
- Scan the **parameter names**, not the values, to see which feature is in play.
- Check the **order** of lines when setup is involved: environment variables, instrumentation, clients.
- Match each option against the code, rather than reasoning from the options alone.
- Don't debug style. Indentation and variable names are almost never the point.

> ON THE EXAM
>
> If two options differ by one parameter, that parameter is the question.

> A snippet builds a keyless client against the v1 endpoint, then calls responses.create with model set to 'gpt-4.1'.
>
> The Foundry project has one deployment, named 'chat-prod'. Every call returns 404.

What should be changed?

A Set model to 'chat-prod', the deployment name in this project.

B Add an api-version query parameter to the client's base URL.

C Switch from Entra ID authentication to an API key for the resource.

D Deploy gpt-4.1 again under its own name so the model name resolves.

◆ PAUSE AND DECIDE

What should be changed?

A Use the deployment name.

model takes the deployment name; 'gpt-4.1' was never deployed here.

B Add api-version.

The v1 endpoint doesn't need a dated api-version.

C Switch to a key.

Auth failures return 401 or 403, not 404.

D Redeploy under the model name.

Works by accident, but it's a deployment change to fix a one-word code bug.

THE SIGNAL

404 with a valid endpoint points at the model parameter nearly every time.

> A team upgrades a Chat Completions call to a reasoning model deployment. The code keeps temperature at 0.2 and max_tokens at 800.
>
> Every request now fails with an unsupported parameter error.

What should the code become?

A temperature 0, max_tokens 800, plus a retry on the error.

B top_p 0.2 instead of temperature, keeping max_tokens as it is.

C temperature 0.2 kept, with max_tokens raised to 4,000.

D No temperature, and max_completion_tokens instead of max_tokens.

◆ PAUSE AND DECIDE

What should the code become?

A temperature 0 and a retry.

Unsupported at any value, and retrying an invalid request just repeats it.

B top_p instead.

Also unsupported, and max_tokens still fails.

C Bigger max_tokens.

The parameter itself is unsupported, whatever its value.

D Drop temperature; max_completion_tokens.

Removes both unsupported parameters and uses the cap reasoning models accept.

THE SIGNAL

Two unsupported parameters, one error. Reasoning models take max_completion_tokens in Chat Completions.

> A search call sets search_text, a vector query with k_nearest_neighbors of 3, query_type 'semantic' and a valid semantic configuration, and top of 10.
>
> Relevance is barely better than before the semantic ranker was enabled.

What should be changed first?

A Remove search_text, so the query becomes a pure vector search.

B Raise k_nearest_neighbors to 50, so the ranker has candidates.

C Raise top to 50, so more results are returned to the caller.

D Remove query_type 'semantic', since it isn't improving results.

◆ PAUSE AND DECIDE

What should be changed first?

A Drop search_text.

Removes keyword recall, giving the ranker even less to work with.

B k to 50.

Microsoft's guidance: feed the semantic ranker 50 candidates.

C top to 50.

Changes how many results are returned, not how many are reranked.

D Remove semantic ranking.

Abandons the feature instead of supplying it with candidates.

THE SIGNAL

k and top are different numbers. k feeds the ranker; top returns results.

> A migrated function calls client.images.generate on a GPT-image deployment, then reads result.data[0].url and downloads it.
>
> The download always fails, although images are generated and billed.

What should the code do instead?

A Poll the job until the image URL becomes available, then download it.

B Set response_format to 'url' explicitly in the generate call.

C Decode result.data[0].b64_json, because these models return base64.

D Set output_format to 'png' so the service returns a hosted file.

◆ PAUSE AND DECIDE

What should the code do instead?

A Poll for a URL.
Image generation isn't a job, and no URL is ever produced.

B Set response_format.
Unsupported on GPT-image models.

C Decode b64_json.
GPT-image models return base64 image data only.

D output_format png.
Chooses the file type inside the base64 payload, not a hosted URL.

THE SIGNAL

Billed but unusable output usually means the response was read in the wrong shape.

Reading code in five lines

- **Client, operation, parameters, fault.** In that order, every time.
- **model= is a deployment name.** 404 with a good endpoint means check it.
- **Reasoning models** reject temperature and max_tokens; **GPT-image** returns base64.
- **k feeds the ranker; top returns results.** Different numbers, different jobs.
- **Setup order matters**: environment variables before instrumentation, clients after.

> ON THE EXAM
>
> Next, module 36: exam mechanics, the last skills module before the practice exams.